

Fase 5 Evaluación Final POA**Estudiante**

Mariela María Márquez Narváez

Curso:

213022

Grupo:

213022_65

Tutor

Eder Leandro Cosme

Universidad Nacional Abierta y a Distancia UNAD
Escuela de Ciencias Básicas, Tecnología e ingeniería ECBTI
Ingeniería Multimedia
2026

Introducción

En el desarrollo de esta actividad se realizó una solución básica en Python para calcular las horas trabajadas de varios empleados durante una semana laboral. Para la solución del problema se utilizaron matrices, funciones, estructuras condicionales y ciclos repetitivos, permitiendo calcular el total de horas semanales y clasificar si el trabajador tiene sobretiempo o un horario estándar.

El desarrollo de esta actividad permitió fortalecer los conocimientos adquiridos durante el curso de Fundamentos de Programación y aplicar los conceptos básicos de programación estructurada mediante el lenguaje Python.

Desarrollo

Objetivo general

Desarrollar un programa en Python que permita calcular las horas trabajadas semanalmente por varios empleados utilizando matrices, funciones, ciclos y estructuras condicionales.

Objetivos específicos

- Implementar una matriz para almacenar la información de los empleados.
- Utilizar funciones para calcular el total de horas trabajadas.
- Aplicar estructuras condicionales para identificar el sobretiempo.
- Mostrar los resultados obtenidos mediante la ejecución del programa.

Problema seleccionado

Problema 5:

Una matriz registra las horas trabajadas por un equipo durante la semana: [Nombre del Recurso, Lunes, Martes, ..., Viernes].

Se requiere calcular el total de horas semanales por persona y clasificar la jornada como “Sobretiempo” si supera las 40 horas o “Horario Estándar” si no excede dicho límite.

Explicación del programa

Entrada

El programa recibe una matriz con los nombres de los empleados y las horas trabajadas de lunes a viernes.

Proceso

Se utiliza una función para sumar las horas trabajadas de cada empleado. Posteriormente, mediante una estructura condicional, se determina si el trabajador tiene sobretiempo o un horario estándar.

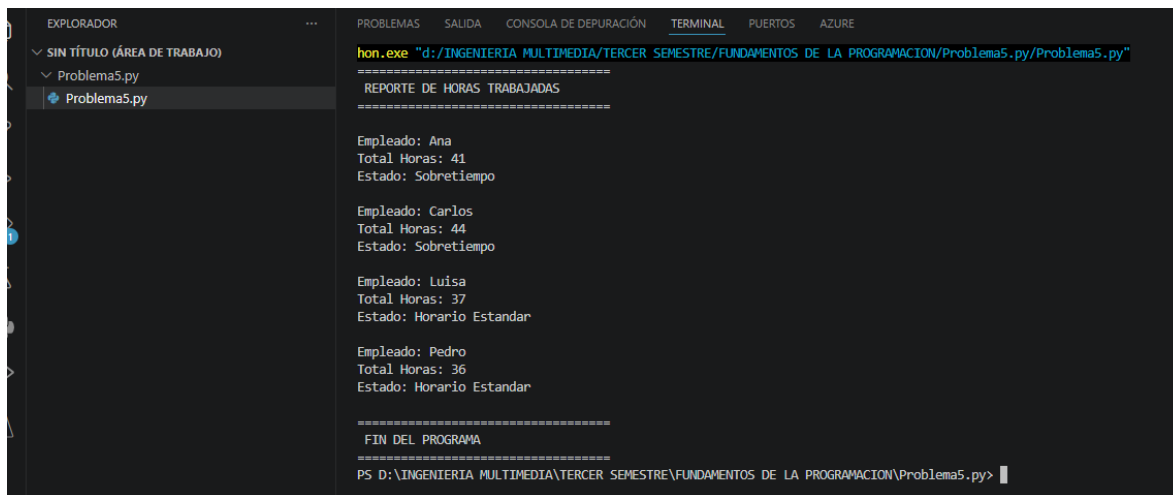
Salida

El programa muestra el nombre del empleado, el total de horas trabajadas y la clasificación de la jornada laboral.

Código Python:

```
# =====  
# FASE 5 - EVALUACION FINAL POA  
# Problema 5  
# =====  
  
empleados = [  
    ["Ana",8,8,9,8,8],  
    ["Carlos",10,9,8,9,8],  
    ["Luisa",7,8,7,8,7],  
    ["Pedro",9,9,9,9],  
]  
  
def calcular_horas(horas):  
  
    total = sum(horas)  
  
    if total > 40:  
        estado = "Sobretiempo"  
    else:  
        estado = "Horario Estandar"  
  
    return total, estado  
  
print("=====")  
print(" REPORTE DE HORAS TRABAJADAS ")  
print("=====")  
  
for empleado in empleados:  
  
    nombre = empleado[0]  
    horas = empleado[1:]  
  
    total, estado = calcular_horas(horas)  
  
    print("\nEmpleado:", nombre)  
    print("Total Horas:", total)  
    print("Estado:", estado)  
  
print("\n=====")  
print(" FIN DEL PROGRAMA ")  
print("=====")
```

Resultados obtenidos:



```
hlon.exe "d:/INGENIERIA MULTIMEDIA/TERCER SEMESTRE/FUNDAMENTOS DE LA PROGRAMACION/Problema5.py/Problema5.py"
=====
REPORTE DE HORAS TRABAJADAS
=====

Empleado: Ana
Total Horas: 41
Estado: Sobre tiempo

Empleado: Carlos
Total Horas: 44
Estado: Sobre tiempo

Empleado: Luisa
Total Horas: 37
Estado: Horario Estandar

Empleado: Pedro
Total Horas: 36
Estado: Horario Estandar

=====
FIN DEL PROGRAMA
=====
PS D:\INGENIERIA MULTIMEDIA\TERCER SEMESTRE\FUNDAMENTOS DE LA PROGRAMACION\Problema5.py>
```

Link video:

https://youtu.be/Gy4YcdSkDqA?si=sOhgNcYDWgk_90tg

Link GitHub:

Conclusiones

En esta actividad pude aplicar los conocimientos aprendidos durante el curso utilizando el lenguaje Python para resolver un problema relacionado con las horas trabajadas de varios empleados. A través del uso de matrices, funciones, ciclos y estructuras condicionales, fue posible calcular el total de horas semanales y determinar si existía sobretiempo.

El desarrollo de este programa me permitió fortalecer mis habilidades en programación y comprender mejor cómo solucionar problemas mediante código de una manera organizada y funcional.